16.SELF INSTRUCT

23 March 2026 11:28

Large instruction tuned language models → great at zero shot to new tasks
(good generalization)

Still they depend heavily on human written instructions → limited in quantity, diversity & creativity

SELF INSTRUCT → models instruction following capabilities improve by bootstrapping off their own generations
↳ annotation free method for alignment
↳ semi automated process for instruction tuning a pre trained LM using the instructional signals from the model itself

An iterative bootstrapping algorithm, which starts off with a limited seed set of manually-written tasks that are used to guide the overall generation

phase I → model prompted to generate instructions for tasks → helps in more broad coverage
instructions being created that define new tasks
↓
the framework also creates input output instances for these new tasks
(can be used for supervising during instruction tuning)
↓
finally low quality or repeated instructions are filtered out before adding remaining to the task pool

generated tasks deviate from distribution of typical NLP tasks

Challenges for humans → creativity for novel tasks
expertise for writing solutions to each task

Instruction Data

I_t → set of instructions
each instruction defines a task 't' in natural language

Task t has $n_t \geq 1$ input-output instances $\{(X_{t,i}, Y_{t,i})\}_{i=1}^{n_t}$

$$M(I_t, X_{t,i}) = Y_{t,i} \text{ for } i \in (1 \quad n_t)$$

No strict boundaries, allow only instructions without input X

Automatic Instruction Data Generation

→ Generating task instructions
→ Determining if the instruction represents a classification task
→ Instance generation with either an input first/output first approach
→ Filtering low quality data

Input first approach → LM comes up with input fields first based on the instructions and then produce the corresponding output
↳ bias towards one label in classification tasks

Output first approach → for classification tasks
↳ first generate class labels
↳ condition the input generation on each class label

Output first approach → for classification tasks
↳ first generate class labels
↳ condition the input generation on each class label

→ New instruction is only added to the pool only when its ROUGE-L similarity with any existing instruction is less than 0.7
→ exclude instructions containing specific keywords like (image, picture, graph)
→ instances that are similar to input, longer or too short etc are excluded

Finetuning

Concatenate Instruction & Instance input as prompt and train the model to generate instance output in a standard supervised way
Multiple templates to make model robust

Quality review questions ?
→ Does the instruction describe a valid task?
→ Is the input appropriate for the instruction?
→ Is the output correct and acceptable response to the instruction & input?
→ All fields are valid?

Model generates (instruction, input, output) triples
Then trains on them as supervised data
↳ This isn't self-training on labels
This is self-expansion of the task distribution

→ Loop is primarily for data growth, not iterative model updating each cycle
↳ In practice → Generate large dataset (multi iteration)
Then single finetuning stage

A data bootstrapping loop

Constraint lens

Input-first → Label imbalance emerges
Output-first → forces coverage over label space

This is distribution control not just bias avoidance

Representation

Before → LM encodes text continuation distribution
After → LM encodes task-conditioned mappings

Instructions become task selectors in embedding space
Model learns conditional manifolds of behaviors

Scaling

Performance ↑ with data
But plateaus (≈ 16K instructions)

Datageneration + Bootstrapping pipeline

Goal = reduce dependence on human instruction data

Core loop = Generate → filter → finetune
(repeat)

Key bottleneck = data diversity + cost

Filtering = implicit objective shaping

replaces reward model (RLHF) human curation

enforces Diversity constraint
↓
ROUGE-L < 0

Modality constr
no image/graph tasks

Format validity constraint

Finetuning Objective

Input = (instruction + input)
Target = Output
Objective = standard next token likelihood

$P(Y \mid I, X)$

Same geometry as instruction tuning

Mechanistic summary

Representation
Instructions → task embeddings
Model learns mapping task → behavior

Objective
Maximize likelihood of instruction assign

Performance ↑ with data
But plateaus (~16K instructions)
→ not purely data limited
Also model capacity / data quality limited

Knowledge distillation
here ↦
Same model generates data → Same model feeds on it
Self distillation in task space

Limitations → Tail phenomenon

Self instruct depends on LMs &
inherit all limitations that
carry over with LMs
→ Dependence on large models
→ Reinforcing LM biases

Filtering isn't cleaning
↳ it is controlling data distribution
+ preventing feedback loops

ROUGE-L → works on sequence overlap
Captures semantic redundancy at surface level

alternatives
(i) embedding based → cosine similarity
(captures semantic similarity)
(ii) Model based scoring
Use LM to score
making it implicit reward model
(proto-RLHF)
(iii) Diversity sampling
Instead of filtering after (enforce diversity during generation)

Gradient flow $(I, X) \to Y$

non differentiable data construction + differentiable training

This system is effectively
optimization over data, not over parameters
Model isn't learning new knowledge

Objective
Maximize likelihood of outputs given (instructions, input)

Information flow
Seed tasks → expanded task space
↓
filtered
↓
training data

Optimization
Standard supervised learning
no RL, no Reward model

Self instruct ≠ self label
(its task generation)

Remove filtering → Distribution collapse
Similar instructions generated
Training data → low entropy
Instruction embeddings collapse to
narrow regions
→ Error amplification
Bad generations recycled → trained on
Bootstrapping effect
Errors become self
reinforcing attractors
→ Format drift
Malformed instructions
inconsistent I/O structure
breaks input/output alignment
& supervised signal quality

Why this doesn't collapse?
(1) stronger Prior (pretraining)
prevents random drift
(2) prompting with seed tasks
keeps generation on manifold
(3) Filtering
Removes low probability junk

It is reorganizing existing knowledge into instruction conditioned behavior

Why this improves zero shot?

(A) coverage of task space

Generated data = Many NLP tasks

(B) Learns task abstraction

Classify

Summarize

generate etc

(C) Instruction → behavior alignment

Instruction becomes latent program selector

Instructions map to regions of behavior space

(D) Format robustness

Improves generalization to unseen prompts

Model learns function over tasks

not just tasks themselves

Core limitation

The model can only generate tasks from its existing distribution

No distbn expansion beyond prior

Rare tasks = low probability

Rare tasks = rarely generated

Contrast with RLHF

↳ introduces external signal

Can push model outside prior

Self-instruct

↳ purely self sampling

↳ stays within prior